DPH Engineering Guide

Delphi Project Handoff

Windows desktop experimental software

Last changed 10 October 2026

For a Delphi developer building maintaining and validating DPH

This guide describes the implemented application and its evidence limits. Use the phase acceptance records for exact measured results and unresolved release gates.

Contents

Architecture and boundaries....3
Build and deployment....3
Input acquisition and project scope....3
Syntax contracts and limits....4
Symbols and typed relationships....4
Control flow and impact....4
SQLite generations and durability....4
Evidence and immutable history....5
Background coordination and cached views....5
FMX navigation and visualization....6
Curated knowledge and review....6
Canonical publication and privacy....6
HTML guides and bundle validation....7
Translation boundary....7
Verification and release discipline....7
Source and revision evidence....8

Architecture and boundaries

DPH is an independent Delphi FMX application. It has no converter runtime or build dependency. The pinned VCL2FMXConverter v6.0.0 corpus is test input only, at commit e1fd10b2d3b06f2fc3f726b8eb7f234a54faf63d. No converter source was copied into the current analyzer. Preserve the original planning documents and global agents.md byte for byte.

The pipeline is selected project, bounded inventory, read-only byte acquisition, lexical and conditional processing, Pascal and form IR, semantic relationships and control flow, revision-consistent SQLite capture, cached workspace projection, curated knowledge and derived publication. Contract data remains separate from FMX. Every unsupported construct or unavailable dependency must remain diagnosable rather than being promoted into false completeness.

All application controls are serialized in DPH.UI.Main.fmx and editable in the IDE. List contents and paint-box geometry are data projections. No scanned component class is instantiated. Avoid adding runtime-only controls, Pascal class helpers or an alternative authoritative store without explicit authorization.

Build and deployment

The installed compiler is RAD Studio 37.0.59082.6021. Load C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat before compiling. tools/build-wp0.cmd builds DelphiProjectHandoff.dproj with elevated MSBuild and runs the FMX resource smoke test. Output is bin/Win32/Debug, with DCUs in dcu/Win32/Debug. The initial executable target is Windows Win32.

Deploy the executable, migrations 001_foundation.sql and 002_constraints.sql, and the architecture-matched WebView2Loader.dll. The reviewed Microsoft WebView2 SDK package is pinned to 1.0.4258.31; retain tools/WebView2/LICENSE.txt with binary redistribution. The browser loads the DLL by its absolute executable-folder path and activates Edge only when preview is requested. There is no intended IE fallback.

The repository standard folders include bin, dcu, docs/guides, docs/pdf, export, help, images and icons, samples, source distributions and tools. Reference source and generated test databases remain ignored. Use the explicitly approved Python 3.13 executable for project tooling and the installed LibreOffice, never Microsoft Word.

Input acquisition and project scope

DPH.Project.Inventory follows the selected project, explicit references, uses clauses, include paths and approved external roots. Project XML is passive: targets and imported build scripts are not executed. Canonical paths are project relative and normalized; traversal, ambiguous macros, reparse paths, cycles and oversized acquisition are rejected or diagnosed. A repository-wide count of Pascal files is not the build scope.

DPH.Project.Source opens input read only, checks the final opened Windows handle path, captures bytes, records SHA256 and verifies decoding by round trip. Unicode byte offsets, line endings and explicit legacy encodings remain in the evidence contract. The maximum acquired source is 16 MiB. Binary forms are parsed as data; unsupported resources remain gaps.

A source viewer must re-acquire the live file and match the captured hash before using a cached range. Externally referenced sources use their explicit canonical inventory identity. Never invent a neighboring source path from a cached filename or execute input to fill an analysis gap.

Syntax contracts and limits

DPH.Parse.Model defines IR schema 1, analyzer DPH-WP2-1, token and node limits, source ranges and diagnostics. Lexing preserves original tokens and conditional information. Preprocessing evaluates supported expressions, explicit defines and includes while retaining uncertainty for unknown branches. An unavailable compiler symbol is not automatically a false condition.

The Pascal parser extracts declarations, routines, anonymous methods and supported statements into IR. Declaration and implementation occurrences remain distinguishable before semantic unification. The form parser accepts supported text and binary DFM/FMX structures without loading component packages. The inventory reads relevant project configuration and resources through bounded adapters.

Current bounds include 250000 tokens, 100000 IR nodes, nesting 128 and 10000 diagnostics. Opaque assembler, malformed syntax and unsupported grammar are partial. Tests use exact golden IR, encoding edge cases, cyclic includes, passive malicious imports and changed source. Maintain source byte ranges and diagnostics when extending support.

Symbols and typed relationships

DPH.Analysis.Model defines semantic schema 1 and analyzer DPH-WP3-1. Stable symbol keys contain qualified identity, signature and conditional variant. DPH.Analysis.Symbols unifies supported routine prototypes and bodies, resolves cross-unit declarations, typed receivers and suitable overloads, associates form events and records anonymous callbacks.

Relationship certainty is confirmed_static, possible_dynamic, inferred, runtime_observed or unresolved. Passive scanning never claims runtime_observed. A uses dependency is not a call. Known lexical calls, serialized bindings and declared types carry derivation rules and source ranges. Recognized asynchronous API spelling can establish a possible scheduling relationship without proving execution order.

Method pointers, unresolved receivers, dynamic dispatch, generated types and framework behavior must retain uncertainty. Basic reads, writes, callback assignment, static SQL spelling and report-model rendering are conservative analyses. Renames are distinct identities until continuity is reviewed; offsets alone must not reassign knowledge.

Control flow and impact

DPH.Analysis.Flow builds per-routine control-flow nodes and typed edges for supported sequences, branches, loops, returns and exception/finally paths. Routine identity, source hash, range, node kind and expression remain available to inspectors and exports. Flow is bounded to 100000 nodes, 300000 edges and nesting 128.

Impact traversal is possible reachability through supported relationships. Do not label it proved runtime impact. The UI bounds a displayed graph to 120 nodes and 960 edges, while the stored semantic graph is the fuller evidence model. Uncertain edges remain visually distinct. Unsupported flow does not become a fabricated successful path.

SQLite generations and durability

TDPHKnowledgeStore owns one FireDAC SQLite connection on its creating thread and serializes writer access through a named mutex. Foreign keys and synchronous FULL are enabled. Default journal mode is DELETE;

WAL is an explicit local-store option. Direct cross-thread writes are rejected. SQLite 3.42.0 with FTS5 was measured by the storage tests.

Schema 2 is created through two checksum-verified migrations. Existing migration text is immutable. A newer incompatible schema is rejected safely. Failed upgrade DDL and ledger changes roll back, and an upgrade from an existing schema makes a recoverable pre-migration database backup. Interrupted running generations recover as failed, preserving the previous published head.

ObserveManifest records current fingerprints independently of the capture. BeginGeneration freezes configuration and manifest identity. Syntax and semantic facts are staged in transactions. Before publication, inputs and configuration are revalidated. Only a coherent complete generation becomes the current head. Partial, failed or superseded captures cannot clear the exact-input stale state or overwrite a prior current head.

Evidence and immutable history

Normalized tables include projects, configurations, source files, file revisions, generations, generation files, entities, entity versions, evidence, relationships, component properties, control-flow nodes and edges, knowledge records and revisions, links, review tasks, translations, artifacts, export jobs and audit events. Foreign keys tie evidence and relationships to their producing generation.

Triggers prevent mutation of terminal generations, source revisions, captured facts, configurations and authored knowledge revisions. Evidence ranges must fit the captured source bytes and use the generation analyzer version. Historical file and entity identities cannot be changed casually. A missing source is retained as missing; its historical annotations survive.

Modification detected, analysis stale, knowledge review needed, analysis failed and reassessment requested are separate persistent flags. A UI summary is a projection of these states, not a replacement for them. Parser success must not clear human review.

Background coordination and cached views

DPH.Workspace.Worker is a persistent TThread with bounded request and response queues. It initializes COM for passive MSXML parsing, owns database operations in the worker, and retains analyzer instances across requests. FMX polls responses with a timer, avoiding queued closures that outlive the form.

A stopped Delphi queue can report success while returning an empty item. Poll checks both the wait result and a non-nil response; shutdown tests guard against a destructor loop. Cancellation is checked between stages and marks a staged running generation failed when appropriate. It is not a promise of interruption inside every parser loop.

Workspace projections are managed JSON files in database.dphdb.views named by SHA256 and registered in artifacts. Cached opening verifies the managed path, size, checksum and generation. The default is detached until explicit source attachment. Connected checks compare manifest fingerprints and selection identity every ten seconds. Syntax cache reuse is bounded; semantic recomputation is currently conservative rather than a claim of a minimal incremental semantic solver.

FMX navigation and visualization

DPH.UI.Main binds its published controls to the FMX resource. Explorer queries are paged at 250 rows; data filtering supports kind, text and qualified scope. History and bookmarks use canonical keys, with bookmarks reset on a new captured generation. Inspector output retains source metadata and serialized properties. Source memo navigation is permitted only after live hash validation.

Graph paint boxes draw bounded evidence data with palette and geometry values supplied by design-time style rectangles. Screen reconstruction draws approximate source-derived rectangles without creating components. Its gallery is bounded to 60 forms. Runtime styles, custom painting and inherited resources are not silently fabricated.

Keep lifecycle and ownership explicit: the view owns its JSON document, UI arrays own their records, responses transfer document or knowledge ownership to the main form, and the worker frees unconsumed responses during shutdown. Guard unsaved authored text before changing projects or replacing editor state.

Curated knowledge and review

SaveKnowledge requires title, body, a named author, a supported kind and a captured generation. Existing records require their expected revision and stable title/kind. Each edit appends a needs_review revision, preserves links and opens a pending task. Knowledge bodies are bounded to 1 MiB. Authored text is never regenerated from parser output.

ReviewKnowledge requires the expected current revision, a named reviewer and a matching captured source/configuration. The connected worker observes current source before accepting a review. A missing linked entity cannot be verified under a replacement name. Verification, rejection or retirement appends a revision and records attribution; history remains append only.

Manifest and configuration changes conservatively invalidate active knowledge, derived artifacts and translations. A partial static capture may support review of an attributable statement against available evidence, but its partial coverage remains visible. Local names are not authenticated enterprise identities. Human verification and automatic static derivation remain different evidence categories.

Canonical publication and privacy

DPH.Publish.Model constructs one versioned documentation model from a captured workspace and current curated knowledge. It carries generation, configuration, input/context fingerprints, detached/stale state, symbols, typed relationships, control flow, source references, human history, coverage and explicit unknowns. Full and task-scoped views share those facts.

Task scope selects the requested identities and complete direct incident context. It rejects missing or excluded seeds and rejects an oversized task rather than silently truncating a full export. External source roots and sensitive filename categories are excluded by default. Explicit relative prefix exclusions remove file-derived symbols and relationships. Source snapshots and generated AGENTS.md are absent by default.

Redaction recognizes common credential assignments, private keys, AWS access keys and JWT-shaped tokens throughout string data. It is defense in depth, not a guarantee that every project-specific secret format is recognizable. Require explicit exclusions and a disclosure review for sensitive content. No translation or AI provider is contacted by normal publishing.

HTML guides and bundle validation

DPH.Publish.Export writes a new destination outside source. The offline HTML uses encoded imported text, local search and styles, internal anchors, print CSS and a restrictive content security policy. The WebView2 navigation policy permits only the authorized local index and about:blank, with no arbitrary native bridge. Data does not supply executable markup.

JSONL indexes contain symbols, relationships, forms/events, data usage, configuration, flow nodes, knowledge and evidence references. Manifest and checksum files accompany canonical.json. The validator checks filenames, hashes, expected input fingerprint, canonical identity, source hash/range, generation and cross-reference integrity. A stale manifest is rejected by default. Unsigned hashes establish integrity relative to the index, not author authenticity.

Editable engineering and user DOCX guides derive from the same sections and evidence model. They include title, last changed capture identity and a changeable TOC field. LibreOffice provides companion PDFs; the project rendering tool verifies cached page numbers and renders page images for layout inspection. Missing business procedures stay explicitly absent rather than being invented.

Translation boundary

IDPHTranslationProvider exposes provider identity, version, external status and prose translation. Protected segments retain identifiers, SQL, paths, hashes and protocol keys byte for byte. Cache keys include source content, revision, source/target locales and provider/version. Changed prose produces a fresh pending entry; named review applies to the exact cached translation.

TDPHAtlasAdapter is deliberately unconfigured because the exact Atlas product, transport, licensing and confidentiality have not been selected. External providers require explicit consent. The current adapter throws before transmission, and tests use an offline mock. The offline GTranslate region is disabled, with local bilingual labels and sorting; do not activate network translation merely because a generic widget name appears in documentation.

Verification and release discipline

Run the phase build/test scripts elevated with the installed toolchain. WP0 checks FMX resource ownership; WP1 checks storage, crash recovery and migrations; WP2 checks inventory, encoding, syntax and source safety; WP3 checks semantics and pinned reference traces; WP4 checks workspace/background behavior; WP5 checks immutable authored history and pending review; WP6 checks publication, exclusions, sanitization, hashes and translation protection.

Use exact input hashes before and after analysis, including failures. Preserve original reference and instruction files. Keep tests that demonstrate invariants rather than tests that merely mirror implementation. New source, validation failure or unresolved risk justifies another targeted test; a passing unchanged phase does not need endless repetition.

Before each authorized phase, create the full D: pre-change backup and verify SHA256. Review intended Git changes, commit validated work and push both public and private remotes without force or history rewriting. Remove a stale zero-byte index lock only after verifying no Git process runs. WP7 must publish measured performance, portability, security and acceptance evidence. Independent first-time programmer acceptance and unknown historical business intent cannot be invented by automation.

Source and revision evidence

The following source files and resources define this guide. SHA256 values identify the inspected implementation. Rebuild the guides after validated changes and retain the associated acceptance evidence.

DPH.Analysis.Flow.pas

`7ae133ca49ea4461c3cfa5a2f659bb035acfa463d34668eac091f405eee0fbad`

DPH.Analysis.Model.pas

`63797e406476e2e527b8fd28b75bbccedbe0820657217457e0329252792bd417`

DPH.Analysis.Persistence.pas

`5771a6e098cd57a17d6221474e0864785675b7240875ccc4e4f9ff284a92a6e6`

DPH.Analysis.Symbols.pas

`3af351398e978e521920fbbd33b8da9d35c1d129176b11437ef836231d0b9a31`

DPH.Parse.Forms.pas

`8b0a82186f7831bd1c27bf377409ac6e49b77c228666e6087fca8758c85d4cc7`

DPH.Parse.Lexer.pas

`6ce5bbc66f8170b17eaeda536b8e500ca9992131477c09bc8e1a33bb6dd1c7f0`

DPH.Parse.Model.pas

`5eb5afb595a879cde199cd029349890990cc58cefdea20e643447dca7bca4f4e`

DPH.Parse.Pascal.pas

`e89e29fce1ece85d5a0ecafad0cf397f42d4797e9f12c997a6ffafe19e0db0c8`

DPH.Parse.Preprocessor.pas

`34b7513019a41bc9911babfda72041addd563959ee54c54d0759b8cc9fa7fefc`

DPH.Project.Analysis.pas

`cad87d7a998ace018b3e57e430ce4737e2679adc378091782aefd07685c803a5`

DPH.Project.Inventory.pas

`b430015f92c1860234c0b6e75f3c0df67bd8f59803d7030be500260bd913f1fa`

DPH.Project.Source.pas

`c2e1ff86fc0ee3435be23cb6fd07bd9df6d43c6954def70cca23f2edf62162cd`

DPH.Publish.BrowserPolicy.pas

`6bab01810016e0b5eef4d22b3152ec44f3af1a17f3b05685b6b710ef66a433a3`

DPH.Publish.Export.pas

`753b14650af70cf085e656172e20d3b33a1d3fbe520d10a22b528d521e25a1a0`

DPH.Publish.Model.pas

`213e1203e2d906b4aaed226b0739b45a1231384e78e4bbf1e5a5a4676910dd89`

DPH.Publish.Translation.pas

`d211794576a0e234d1aa8ac347d09f6a981e9590346204ef142608aa5aa400b2`

DPH.Storage.Database.pas

43af9edc4f4ad9e0750696698fb306be5d784297644fe223ae7a9add8f84af11

DPH.Storage.Types.pas

1e83ccbe6868979ff9d763a4f2bf4d64691c2b2343de6717391f03e204d5d219

DPH.UI.Main.pas

e2284ddbeee35f31df6f53eaff372bd334ebea64098d5547a6085daab0da80c4

DPH.Workspace.Model.pas

99b456e51f003453beb53892d419af0bd98b747a08062391e0a709ca803d78c7

DPH.Workspace.Screen.pas

68305e6ec36664e76d05b8c9af21416dd51a32116d47ea55156977b4c10855de

DPH.Workspace.Worker.pas

c5be6e7821b2b857f2d632ab711d72afba96ff777d19f4b072b406c0ccaa3187

DPH.UI.Main.fmx

56f86ff4ce7be48251b1ffe8297226e1beee217dbba64cfd0d7dbe1b1d6c4de6

DelphiProjectHandoff.dpr

3a7069ba071bb567f4d61da03cca38db4a12b1c700bca885c7e2a46a52d71732

DelphiProjectHandoff.dproj

23428c608a2cee9b8e1bb343af3e201abbe3579e3e49bac25a3b773744701839